session 1

Part 1: Environment Setup & OOP Revision

1.1 Work Environment Setup

Ensure you have the following installed and configured:

1. **JDK (Java Development Kit)**: Version 11 or higher (OpenJDK or Oracle JDK).
2. **IDE**: IntelliJ IDEA, Eclipse, or VS Code configured with the Java Extension Pack.
3. **Execution Check**: Verify your setup by opening a terminal/IDE console and running:

```
java -version
javac -version
```

Cost Estimation

- estimate cost by counting the **number of fundamental operations** (e.g., additions, array accesses, loop iterations) executed as a function of the input size N.
- Estimate memory usage as a function of the input size N.

Example 1: Factorial (Iterative)

```
public class Factorial {
    public static long factorial(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }
}
```

Example 2: Fibonacci (Iterative vs. Simple Recursive)

Iterative Fibonacci

```
public static long fibonacciIterative(int n) {
    if (n <= 1) return n;
    long a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        long temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}
```

Simple Recursive Fibonacci

```
public static long fibonacciRecursive(int n) {
    if (n <= 1) return n;
    return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
}
```

Example 3: Nested Loops (Matrix / Pair Checks)

```
public static int countPairs(int[] a) {
    int count = 0;
    int n = a.length;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (a[i] == a[j]) {
                count++;
            }
        }
    }
    return count;
}
```

3B. Triangular Nested Loop (Distinct Pairs)

```
public static int countUniquePairs(int[] a) {
    int count = 0;
    int n = a.length;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] + a[j] == 0) {
                count++;
            }
        }
    }
```

```
    return count;
}
```

Part 3: Empirical Timing in Java

```java
long startTime = System.nanoTime();

// Call the method/algorithm to measure
long result = Factorial.factorial(40);

long endTime = System.nanoTime();
double elapsedTimeInSeconds = (endTime - startTime) / 1_000_000_000.0;

System.out.printf("Elapsed time: %.6f seconds\n", elapsedTimeInSeconds);
```

Part 4: Hands-On Exercise & Experimentation

Task Instructions

1. Implement a timing benchmark comparing `fibonacciIterative(n)` and `fibonacciRecursive(n)` for inputs $N = 10, 20, 30, 40, 45$.
2. Fill out the observation table below with your measured timings.
3. Graph or note how the time increases as N grows for both methods.

Input (N)	Iterative Time (s)	Recursive Time (s)	Theoretical Ratio/Trend Observed
10			
20			
30			
40			
45			

Quick OOP Revision

In this course, we will heavily rely on clean object-oriented design and abstraction.

Key Concepts:

- **Classes & Objects**: A *class* is a blueprint; an *object* is an instance of that blueprint containing state (fields) and behavior (methods).
- **Encapsulation**: Hiding internal implementation details using `private` access modifiers and providing public interfaces (getters/setters/methods).
- **Generics (Crucial for Data Structures)**: Writing code that works with arbitrary data types.
- **Interfaces & Abstract Data Types (ADTs)**: Decoupling *what* an operation does from *how* it is implemented.

Practice Task:

Create `FlexibleArray` class that holds an array of a given starting size which expands or shrinks based on need.

- if the next insert requires a space beyond that of the array expand to double length
- If the latest pop (returns the last item and deletes it) shrinks to less than half the current size then drop the size to half

Discussion & Wrap-Up Questions

1. **Doubling Hypothesis**: When you double the size of input N in an $O(N^2)$ algorithm, by what factor does the runtime increase?
2. **Measurement Noise**: Why might running the same timing test twice yield slightly different results in Java? (Hint: consider JVM Warmup, Garbage Collection, and OS Scheduling).
3. **Memory vs Time**: How does the memory usage (stack depth) differ between iterative and recursive implementations?